

Ein prüfbarer Rechenkernel

Konzept und erster Durchstich

da3Dalus — Rechenkanon

Pfad 1 „Geschwindigkeiten“ · alle Einträge status: draft

18. August 2026

Inhaltsverzeichnis

1. Das Problem	1
2. Die These	2
3. Der Aufbau — drei Ebenen	2
4. Die maschinellen Prüfungen	3
5. Der Durchstich — <code>v_stall</code>	4
6. Was der Durchstich zeigt	5
7. Aufwand und Reichweite	6
8. Was ich nicht behaupte	6
9. Was ich vorschlage	7

1. Das Problem

Diese Anwendung rechnet an 1112 Stellen. Sie ist nicht prüfbar, und der Grund ist nicht mangelnde Sorgfalt.

Jede einzelne Zeile ist richtig. `cl_max = np.max(cl_arr)` ist korrektes Python und korrekte Numerik. `v_stall_ldg = aircraft.get("v_s0_mps")` or `v_stall` ist ein sauberer Rückfall. `compute_vn_curve(rho: float = 1.225)` ist eine legitime Signatur. Wer den Code liest, findet nichts.

Die Fehler liegen zwischen den Rechnungen:

Beobachtung	Form
<code>shear_N</code> wird berechnet, nie gelesen	Knoten ohne ausgehende Kante
<code>g</code> ist elfmal definiert, in zwei Werten	ein Wert, elf Autoritäten
<code>V_stall</code> entsteht in sechs Dateien	eine Größe, sechs Erzeuger
<code>utilisation</code> misst Bandpassung, nicht Festigkeit	Name widerspricht Definition
<code>C_L,max</code> vom schnellsten Punkt berechnet die langsamste Geschwindigkeit	richtige Formel, falsche Bindung

Keiner dieser Befunde ist durch Lesen einer Funktion zu finden. Alle sind sichtbar, sobald man die Rechnung als Graph betrachtet — und entscheidbar erst, wenn es eine Sollaussage gibt, gegen die man prüft.

Vier ADRs dieses Projekts (0019, 0020, 0022, 0023) sind bereits Regeln über genau diesen Graphen. Sie werden verletzt, ohne dass es auffällt, weil das Objekt, über das sie sprechen, nicht existierte.

2. Die These

Ein Register dessen, *was der Code tut*, reicht nicht. Prüfbarkeit entsteht erst durch ein zweites, kleineres Register dessen, *was er tun soll* — und durch die Abbildung zwischen beiden.

Daraus folgen drei Behauptungen, die der Durchstich belegen oder widerlegen soll:

B₁ — Die Sollmenge ist klein genug zum Lesen. 1112 Knoten fallen auf eine Größenordnung von 200 Aussagen zusammen, ein Pfad auf 25–45. Das ist prüfbar; 1112 Knoten sind es nicht.

B₂ — Ein freigegebenes Gesetz ist ein Testorakel. Ohne Sollaussage gibt es nur Regressionstests, die den heutigen Zustand samt seiner Fehler zementieren. Ein *fachlicher* Test braucht etwas, wogegen er behauptet.

B₃ — Ein Teil der Prüfung ist maschinell. Dimensionen, Provenienz, Übereinstimmung zwischen Rechenwegen: das braucht kein Urteil. Menschliches Urteil bleibt nötig, aber nur für das, was wirklich Urteil ist.

3. Der Aufbau — drei Ebenen

3.1 Warum nicht eine

Ein reines Formelregister versteckt den wichtigsten Unterschied. Zwei Stellen, die dieselbe Größe verschieden rechnen, sind ein Konflikt. Zwei Stellen, die *dieselbe* Formel mit verschiedenen Eingaben rechnen, sind eine Anwendung — völlig normal. In einem Formelregister sehen beide identisch aus.

3.2 Die Ebenen

Ebene	Aussage	Freigabe prüft
Größe	Symbol, Einheit, Bedeutung, Rolle (Eingabe / abgeleitet / Ausgabe)	Ist es <i>eine</i> Größe?
Formel	eine Beziehung zwischen Größen, in Symbolen	Quelle · Maßstab 0,5–15 kg · Dimensionen
Anwendung	die Formel an ein Problem gebunden, mit Bedingung	Ist die Bindung die richtige?
Vorbedingung	was für die Eingaben gelten muss, damit die Bindung etwas bedeutet	Hält sie? Welcher Test klärt es?

Die vierte Ebene ist die, an der die Fehler dieses Codes tatsächlich sitzen. Sie kam zuletzt hinzu und war der Punkt, an dem das Modell erst trug.

3.3 Die vier Formen

„Zwei Erzeuger einer Größe“ verbirgt vier Situationen, und nur die letzte ist eine Entscheidung:

Form	Bedeutung	Konsequenz
Gesetz	eine Beziehung, überall angewendet, wo sie gebraucht wird	einmal freigeben
Rechenweg	dieselbe Größe auf zwei Wegen — geschlossen und numerisch	✓ erzeugt einen Test: sie müssen übereinstimmen
Näherung	eine Faustformel dort, wo ein Gesetz hingehört	! kennzeichnen, nie <i>als</i> Gesetz freigeben
Konflikt	zwei Gesetze beanspruchen eine Größe	! entscheiden

Eine fünfte Lage betrifft nicht den Kanon, sondern den Code: ein Gesetz, uneinheitlich implementiert. Das ist ein Implementierungskonflikt, und die Anwendungsebene löst ihn auf.

3.4 Freigabe

draft -> approved -> superseded. Zwei Stufen, in dieser Reihenfolge:

Zuerst die Formel — einmal, und alle Anwendungen erben es. Dann die Anwendung — je Bindung, je Bedingung.

Eine Regel ordnet die Arbeit: eine Formel ist erst freigebbar, wenn ihre Eingänge freigegeben sind. Daraus folgt die Traversierung von den Eingabeparametern zur Ausgabe — und das einzige ehrliche Fortschrittsmaß: *welcher Anteil der Pfade zu einer nutzersichtbaren Zahl ist freigegeben.*

4. Die maschinellen Prüfungen

Was ohne Urteil entschieden werden kann, wird ohne Urteil entschieden.

Dimensionsalgebra (`scripts/dimensions.py`, `scripts/check_canon.py`). Einheiten werden in einen SI-Dimensionsvektor plus Längenmaßstab zerlegt. Der Maßstab ist kein Luxus: Dieses Projekt führt Millimeter in einem Meter-Modell (ADR 0001), und eine reine Dimensionsprobe lässt mm gegen m durch. Winkel bekommen ein eigenes Fach, damit Grad nicht auf Bogenmaß trifft.

Provenienz. Jeder Knoten nennt `file:line` plus Symbolnamen. Ein Skript prüft, ob das Symbol dort noch steht. Das fängt die häufigste Verrottung — umbenannt, verschoben, gelöscht — für fast nichts.

Rechenwegprüfung. Wo zwei Wege dieselbe Größe beanspruchen, ist die Behauptung „sie stimmen überein“ ein Test, kein Urteil.

Was daraus schon herausfiel, bevor ein Mensch gelesen hatte:

16 von 20 Lastvielfachen-Knoten tragen die Einheit **g** — eine Beschleunigung. Ein Lastvielfaches ist $n = L/W$, Kraft durch Kraft, also dimensionslos. N/N geht nicht gegen m/s^2 auf. Dafür braucht man keine Aerodynamik.

5. Der Durchstich — `v_stall`

Ein Pfad von den Eingabeparametern bis zu einer nutzersichtbaren Zahl, auf allen Ebenen ausgearbeitet. Vollständig in `formulas/stall-speed.md`; hier die Kette.

5.1 Der Pfad

Eingaben	$m \text{ [kg]} \cdot S_{\text{ref}} \text{ [m}^2\text{]} \cdot C_{L,\text{max}} \text{ [-]} \cdot h \text{ [m]} \cdot g \text{ [m/s}^2\text{]}$		
Gesetz 1	$W = m \cdot g$	[ok] Dimensionen: N ok	
Gesetz 2	$\rho = \rho_{\text{ISA}}(h)$	[-] Verfahren, kein Gesetz	
Gesetz 3	$V_S = \text{sqrt}(2W / (\rho \cdot S_{\text{ref}} \cdot C_{L,\text{max}}))$	[ok] Dimensionen: m/s ok	
Anwendungen	$v_{\text{stall_clean_mps}} \leftarrow C_{L,\text{max,clean}}$	(immer)	
	$v_{\text{stall_takeoff_mps}} \leftarrow C_{L,\text{max,takeoff}}$	(nur mit Klappe)	
	$v_{\text{stall_landing_mps}} \leftarrow C_{L,\text{max,landing}}$	(nur mit Klappe)	
Ausgaben	V-n-Diagramm · Geschwindigkeitsleiste · Missions-KPI · Startstrecke		

5.2 Ebene Formel — trägt

Quelle • Sadraey §4.3.2 Gl. 4.30; Scholz *o5_PreliminarySizing* §5.1 für die Landekonfiguration; Anderson *FoA* 6e §4.13 für die zugrundeliegende Aussage.

Dimensionen • $\sqrt{N / (\text{kg} \cdot \text{m}^{-3} \cdot \text{m}^2 \cdot 1))} \rightarrow \text{m/s}$. Geht auf.

Maßstab 0,5–15 kg — die Gleichung ist exakt; die Einschränkung liegt in der Eingabe ($\rightarrow$ 5.4).

5.3 Ebene Anwendung — deckt einen Widerspruch auf

Der Code behandelt die Konfigurationen als drei Implementierungen statt als drei Bindungen. Folge, in einer Funktion, sieben Zeilen auseinander:

```
361: cl_max_ldg = aircraft.get("cl_max_landing") or cl_max_base * ldg_factor # bis ×1,6
368: v_stall_ldg = aircraft.get("v_s0_mps") or v_stall # der SAUBERE Wert
```

Dieselbe Funktion nimmt an, die Klappen wirken (für $C_{L,\text{max}}$), und gleichzeitig, sie wirken nicht (für V_S). Auf dem Rückfallpfad ist die Anfluggeschwindigkeit um $\sqrt{1,6} = 26\%$ zu hoch, während die Landestrecke mit dem erhöhten $C_{L,\text{max}}$ gerechnet wird — zwei Effekte in entgegengesetzte Richtung.

Die Anwendungsebene beseitigt das strukturell, nicht durch eine Korrektur: Es gibt keinen Rückfallpfad mehr, weil `v_stall_landing_mps` gar nicht existiert, wenn keine Klappe da ist — und wenn eine da ist, kommt sie aus derselben Formel mit `cl_max_landing`.

(Von 29 Flugzeugen im Bestand haben 2 eine Klappe. Die Bedingung an der Anwendung ist also nicht theoretisch.)

5.4 Ebene Vorbedingung — der eigentliche Ertrag

`cl_max` — • verletzt.

Anforderung: ermittelt bei der Reynoldszahl des Abrisses.

Warum es die Antwort entscheidet: Bei niedriger Re bleibt die Grenzschicht länger laminar, löst gegen den Druckanstieg ab und bildet eine laminare Ablöseblase, die die Saugspitze kappt. Lennon dokumentiert für NACA 0012 einen Abfall von $C_{L,max}$ 1,55 $\rightarrow$ 0,83 über den Modell-Re-Bereich, Abrisswinkel $17^\circ \rightarrow 10^\circ$. Da $V_S \sim C_{L,max}^{(-1/2)}$, entscheidet die Bindung die Antwort.

Konsequenz: V_S ist im Modellmaßstab eine implizite Gleichung — V_S hängt von $C_{L,max}(Re)$ ab, und Re hängt von V_S ab. Es braucht einen Fixpunkt.

Im Code: `_fine_sweep_cl_max` legt ein Gitter Geschwindigkeit $\times$ Anstellwinkel von `max(v_cruise * 0,5, 3, 0)` bis `v_max` und nimmt dann `cl_max = np.max(cl_arr)` — das Maximum über alle Geschwindigkeiten. NeuralFoil modelliert die Re -Abhängigkeit korrekt; das Maximum verwirft sie und wählt den schnellsten Abtastpunkt, um die Geschwindigkeit am langsamsten Punkt zu berechnen.

Richtung: unsicher — $C_{L,max}$ zu hoch, V_S zu niedrig gemeldet.

Nebenbefund: Ein Trainer hat $V_{cruise}/V_S \approx 2,2$, also $V_S \approx 0,45 \cdot V_{cruise}$ — knapp unterhalb der unteren Gittergrenze. Der Punkt, um den es geht, wird oft gar nicht abgetastet.

Test, der es klärt: $C_{L,max}(V)$ sweepen statt `max` über alles, unteres gegen oberes Ende vergleichen. Laufen sie auseinander, ist die Fixpunktiteration nötig; laufen sie nicht auseinander, ist die heutige Vereinfachung belegt statt angenommen.

`rho` — • verletzt. `compute_vn_curve(rho: float = 1.225)`, und der einzige Aufrufer übergibt sie nie. Das V - n -Diagramm ist immer ein Meereshöhenergebnis, während die Geschwindigkeitspolare ρ auf Höhe nimmt: zwei Abrissgeschwindigkeiten für ein Flugzeug.

6. Was der Durchstich zeigt

Zu B1 — die Sollmenge ist klein genug. 157 Registerknoten $\rightarrow$ 65 Größen, 46 Formeln (39 Gesetze, 3 Rechenwege, 4 Näherungen). Zehn gemeldete Konflikte fallen auf einen echten zusammen (`C_D0`); drei waren falsch zusammengelegte Größen (Istwert gegen Auslegungsgrenze), zwei Rechenweg-plus-Näherung. ✓

Zu B2 — Testorakel. Aus dem Pfad fallen drei Tests ohne Zusatzarbeit: die Übereinstimmung der beiden V_{md} -Rechenwege, der $C_{L,max}(V)$ -Vergleich, und die Dimensionsprobe als Freigabeter. ✓

Zu B3 — maschinelle Prüfung. 29 von 46 Formeln dimensional geprüft, 11 als Verfahren erkannt, 2 mit offener Voraussetzung. Die g -Einheit-Sache fiel ohne Domänenwissen heraus. ✓

Und der Befund, auf den es ankommt: Die $C_{L,max}(Re)$ -Verletzung ist mit keiner der anderen Methoden dieses Projekts gefunden worden — nicht durch Lesen, nicht durch den Provenienz-Audit (1043

bestätigte Knoten, dieser war einer davon: die Zeile ist korrekt), nicht durch den Defekt-Schwarm. Sie wurde sichtbar, als die Formel eine Sollaussage hatte und die Frage lautete: *unter welcher Bedingung bedeutet die Bindung, was sie behauptet?*

7. Aufwand und Reichweite

Register aufbauen (einmalig, ganze App)	22 Agenten, 1112 Knoten
Provenienz-Audit	25 Agenten, 1043/1086 bestätigt, 38 korrigiert
Kanon Pfad 1	3 Agenten + Nacharbeit → 46 Formeln
Freigabe Pfad 1	noch nicht begonnen — 0 von 46

Für die restlichen Pfade (Struktur, Masse, Antrieb, Stabilität) ist das Register bereits da; es fehlt je Pfad der Kanon-Durchlauf. Größenordnung: 150–250 Formeln insgesamt, verteilt auf fünf Pfade.

8. Was ich nicht behaupte

Der Kanon ist unbewiesen, solange kein Pfad freigegeben ist. Null von 46 Formeln tragen approved. Das Konzept steht; der Beleg steht aus.

Ein Kanon, der driftet, ist schlimmer als keiner, weil er zitiert wird. Die Gegenmaßnahmen sind mechanisch (Symbolprüfung in CI, „Implementierung ohne kanonische Formel ist ungeprüfter Code“), aber keine davon fängt jede inhaltliche Drift.

Die Agentenqualität ist Entwurfsqualität. Der Prüfagent fand in der vorgeschlagenen Zuordnung 1 Fehlzuordnung und 5 unerklärte Abweichungen. Ein Vorschlag ist kein Kanon.

Der Defektschwarm hat sich nicht bewährt. 638 Behauptungen, 46 % bestätigt trotz Anweisung zu widerlegen. Diese Zahlen stehen ausdrücklich als ● im Register und dürfen kein Refactoring begründen. Der Kanonweg ersetzt sie nicht zufällig, sondern weil er das liefert, was sie nicht konnten: eine Sollaussage.

Die Maßstabsfrage ist offen und größer als der Kanon. 147 Konstanten tragen eine Herkunft aus Verkehrsflugzeug-Literatur. Ob sie bei 0,5–15 kg gelten, entscheidet keine Formelprüfung, sondern Messung am Flugzeug.

9. Was ich vorschlage

Der Durchstich ist der Prüfstein, nicht das Ergebnis. Die Frage an Sie ist nicht, ob `V_stall` richtig ist — sondern ob diese Form der Darstellung eine Freigabe trägt.

Konkret zu entscheiden:

1. Reichen die vier Ebenen, oder fehlt eine?
2. Ist die Freigabe-Checkliste je Formel das richtige Maß — oder zu grob, zu fein?
3. Trägt die Trennung Formel / Anwendung / Vorbedingung auch bei einem Pfad, der weniger sauber ist als dieser?

Danach: $C_L, \max(V)$ messen, damit die erste Vorbedingung von „*verletzt, Ausmaß unbekannt*“ auf „*verletzt, Ausmaß X %*“ geht — das ist der Unterschied zwischen einem Ticket und einer Priorität.